

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

А.В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

КУРСОРЫ

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

И.А. Лаврешин
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Освоить использование курсоров в PostgreSQL при решении прикладных задач на учебной базе данных кадрового учёта.

2 Задание и исходные данные

Реализовать ПЗ или ХП или Т, использующие курсоры (ISO Syntax).

Продемонстрировать различия в работе статических и динамических курсоров, а также курсоров keyset (Transact-SQL Extended Syntax).

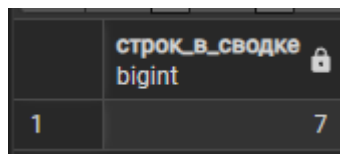
Используя директивы update и delete с условиями where current of самостоятельно реализовать примеры изменения и удаления записей посредством курсора.

3 Ход выполнения задания

В файле 01_aux_stats_table.sql создана таблица lr8_priem_stats для хранения сводной статистики по приёмам на работу в разрезе подразделения и должности (количество приёмов и число совместителей). Первичный ключ задан по паре (ид_подразделения, ид_должности). После выполнения скрипта проверена структура таблицы и подтверждено, что на момент создания записей в ней нет. Результаты работы представлены на рисунках 1 – 2.

```
CREATE TABLE IF NOT EXISTS lr8_priem_stats (  
  ид_подразделения int NOT NULL,  
  ид_должности int NOT NULL,  
  всего_приемов int NOT NULL DEFAULT 0,  
  совместителей int NOT NULL DEFAULT 0,  
  PRIMARY KEY (ид_подразделения, ид_должности)  
);  
  
COMMENT ON TABLE lr8_priem_stats IS  
  'ЛР8: сводка по приёмам (подразделение x должность), заполняется курсором.';
```

Рисунок 1 – Процедура “lr8_priem_stats”



	строк_в_сводке
	bigint
1	7

Рисунок 2 – Результаты работы программы

В файле 02_proc_fill_stats_iso_cursor.sql разработана хранимая процедура lr8_fill_priem_stats . Процедура очищает таблицу lr8_priem_stats, объявляет курсор cur_stats по запросу с группировкой данных таблицы прием, открывает его и в цикле последовательно извлекает строки командой FETCH. Каждая полученная строка заносится в сводную таблицу. По завершении обхода курсор закрывается. Данный фрагмент соответствует ISO-синтаксису работы с курсором (DECLARE, OPEN, FETCH, CLOSE), рассматриваемому в п. 8.1 методички. Процедура вызвана командой CALL; в отчёт включён результат выборки из lr8_priem_stats с присоединением справочников подразделение и должность. Результаты работы представлены на рисунках 3 – 4.

```

CREATE OR REPLACE PROCEDURE lr8_fill_priem_stats()
LANGUAGE plpgsql
AS $$
DECLARE
    v_подр int;
    v_дол int;
    v_всего int;
    v_совм int;
    cur_stats CURSOR FOR
        SELECT
            пр.ид_подразделения,
            пр.ид_должности,
            COUNT(*)::int,
            COUNT(*) FILTER (WHERE пр.совместительство = B'1')::int
        FROM прием пр
        GROUP BY пр.ид_подразделения, пр.ид_должности
        ORDER BY пр.ид_подразделения, пр.ид_должности;
BEGIN
    DELETE FROM lr8_priem_stats;

    OPEN cur_stats;
    LOOP
        FETCH cur_stats INTO v_подр, v_дол, v_всего, v_совм;
        EXIT WHEN NOT FOUND;

        INSERT INTO lr8_priem_stats (ид_подразделения, ид_должности, всего_приемов, совместителей)
        VALUES (v_подр, v_дол, v_всего, v_совм);
    END LOOP;
    CLOSE cur_stats;
END;
$$;

```

Рисунок 3 – Процедура “lr8_fill_priem_stats”

	ид_подразделения integer	подразделение character varying (200)	ид_должности integer	должность character varying (150)	всего_приемов integer	совместителей integer
1	2	Цех механической обработ...	1	Инженер-конструктор 1 к...	1	0
2	2	Цех механической обработ...	3	Техник	3	0
3	2	Цех механической обработ...	90	LR6 должность id 90	1	0
4	3	Отдел главного конструктор...	1	Инженер-конструктор 1 к...	2	0
5	3	Отдел главного конструктор...	2	Инженер-технолог	1	0
6	3	Отдел главного конструктор...	3	Техник	4	0
7	4	Сектор расчётов ОГК	1	Инженер-конструктор 1 к...	1	1

Рисунок 4 – Результаты работы программы

В файле 03_proc_update_where_current_of.sql создана процедура lr8_reset_sovmest_old_priem. Для строк таблицы прием с датой приёма ранее 2016 года и установленным признаком совместительства объявляется курсор с опцией FOR UPDATE. В цикле выполняется извлечение очередной строки и её изменение оператором UPDATE ... WHERE CURRENT OF, то есть обновляется именно текущая строка курсора. На примере записи с идентификатором 8 показано состояние поля совместительство до и после вызова процедуры. Результаты работы представлены на рисунках 5 – 6.

```

CREATE OR REPLACE PROCEDURE lr8_reset_sovmest_old_priem()
LANGUAGE plpgsql
AS $$
DECLARE
    v_ид int;
    v_n int := 0;
    cur_upd CURSOR FOR
        SELECT пр.ид
        FROM прием пр
        WHERE пр.дата_приема < DATE '2016-01-01'
            AND пр.совместительство = B'1'
        FOR UPDATE OF пр;
BEGIN
    OPEN cur_upd;
    LOOP
        FETCH cur_upd INTO v_ид;
        EXIT WHEN NOT FOUND;

        UPDATE прием
        SET совместительство = B'0'
        WHERE CURRENT OF cur_upd;

        v_n := v_n + 1;
    END LOOP;
    CLOSE cur_upd;

    RAISE NOTICE 'lr8: UPDATE WHERE CURRENT OF — изменено строк: %', v_n;
END;
$$;

```

Рисунок 5 – Процедура “lr8_reset_sovmest_old_priem”

	всего_сброшено_совместителей	
	integer	
1		0

Рисунок 6 – Результаты работы программы

В файле 04_proc_delete_where_current_of.sql реализована процедура lr8_delete_demo_priem_cursor, демонстрирующая удаление через курсор. Предварительно в таблицу прием вставлены демонстрационные строки с идентификаторами 900 и 901. Процедура объявляет курсор FOR UPDATE по приёмам с ид ≥ 900 и в цикле удаляет каждую текущую строку командой DELETE ... WHERE CURRENT OF. Контрольным запросом подтверждено

отсутствие демонстрационных записей после выполнения. Результаты работы представлены на рисунках 7 – 8.

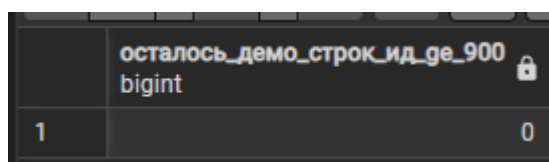
```
CREATE OR REPLACE PROCEDURE lr8_delete_demo_priem_cursor()
LANGUAGE plpgsql
AS $$
DECLARE
    v_ид int;
    v_n int := 0;
    cur_del CURSOR FOR
        SELECT пр.ид
        FROM прием пр
        WHERE пр.ид >= 900
        FOR UPDATE OF пр;
BEGIN
    OPEN cur_del;
    LOOP
        FETCH cur_del INTO v_ид;
        EXIT WHEN NOT FOUND;

        DELETE FROM прием
        WHERE CURRENT OF cur_del;

        v_n := v_n + 1;
    END LOOP;
    CLOSE cur_del;

    RAISE NOTICE 'lr8: DELETE WHERE CURRENT OF – удалено строк: %', v_n;
END;
$$;
```

Рисунок 7 – Процедура “lr8_delete_demo_priem_cursor”



осталось_демо_строк_ид_ge_900	
bigint	
1	0

Рисунок 8 – Результаты работы программы

В файле 05_demo_cursor_types.sql выполнено сравнение вариантов использования курсоров в PostgreSQL. В отдельных блоках PL/pgSQL показаны: курсор без опции SCROLL (движение только вперёд); курсор SCROLL с возможностью FETCH PRIOR; поведение, аналогичное статическому курсору Transact-SQL — обход данных, заранее скопированных во временную таблицу; поведение, близкое к динамическому курсору — чтение непосредственно из основной таблицы штатное_расписание с учётом изменений в текущей сессии. Результаты демонстраций сведены во

временную таблицу и выведены в виде итоговой таблицы, дополненной пояснением соответствия терминов T-SQL (STATIC, KEYSET, DYNAMIC) и средств PostgreSQL. Результаты работы представлены на рисунке 9.

	tsql text	смысл text
1	T-SQL STATIC	снимок в tempdb, не видит чужие изменения
2	T-SQL KEYSET	фиксированы ключи, данные строк могут меняться
3	T-SQL DYNAM...	каждый FETCH видит актуальные данные
4	PostgreSQL	NO SCROLL / SCROLL / снимок TEMP / FOR UPDATE – см. демо в...

Рисунок 9 – Результаты работы программы

В файле 06_trg_refresh_stats_cursor.sql созданы функция lr8_fn_refresh_priem_stats и триггер lr8_trg_refresh_priem_stats на таблице прием. Триггер срабатывает после операций INSERT, UPDATE и DELETE (уровень FOR EACH STATEMENT) и пересчитывает содержимое lr8_priem_stats тем же курсорным алгоритмом, что и в процедуре lr8_fill_priem_stats. Проверено наличие триггера в каталоге information_schema и заполненность сводной таблицы после вызова процедуры заполнения. Результаты работы представлены на рисунках 10 – 11.


```

CREATE OR REPLACE FUNCTION lr8_fn_refresh_priem_stats()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_подр int;
    v_дол int;
    v_всего int;
    v_совм int;
    cur_ref CURSOR FOR
        SELECT
            пр.ид_подразделения,
            пр.ид_должности,
            COUNT(*)::int,
            COUNT(*) FILTER (WHERE пр.совместительство = B'1')::int
        FROM прием пр
        GROUP BY пр.ид_подразделения, пр.ид_должности;
BEGIN
    DELETE FROM lr8_priem_stats;
    OPEN cur_ref;
    LOOP
        FETCH cur_ref INTO v_подр, v_дол, v_всего, v_совм;
        EXIT WHEN NOT FOUND;
        INSERT INTO lr8_priem_stats (ид_подразделения, ид_должности, всего_приемов, совместителей)
        VALUES (v_подр, v_дол, v_всего, v_совм);
    END LOOP;
    CLOSE cur_ref;
    RETURN NULL;
END;
$$;
DROP TRIGGER IF EXISTS lr8_trg_refresh_priem_stats ON прием;
CREATE TRIGGER lr8_trg_refresh_priem_stats
AFTER INSERT OR UPDATE OR DELETE ON прием
FOR EACH STATEMENT
EXECUTE PROCEDURE lr8_fn_refresh_priem_stats();

```

Рисунок 10 – Процедура “lr8_fn_refresh_priem_stats”

	строка_в_сводке_после_создания_триг
bigint	
1	7

Рисунок 11 – Результаты работы программы

4 Вывод

В лабораторной работе реализованы курсоры в хранимых процедурах: заполнение сводки lr8_priem_stats, изменение и удаление строк через UPDATE/DELETE ... WHERE CURRENT OF, сравнение типов курсоров и пересчёт сводки триггером на таблице прием. Показано, что курсор уместен для построчной логики, а для простых агрегатов эффективнее обычный SQL.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
-- ===== --
-- 01_aux_stats_table.sql
-- ===== --

CREATE TABLE IF NOT EXISTS lr8_priem_stats (
    ид_подразделения int NOT NULL,
    ид_должности int NOT NULL,
    всего_приемов int NOT NULL DEFAULT 0,
    совместителей int NOT NULL DEFAULT 0,
    PRIMARY KEY (ид_подразделения, ид_должности)
);

COMMENT ON TABLE lr8_priem_stats IS
    'ЛР8: сводка по приёмам (подразделение × должность), заполняется
    курсором.';

-- ===== --
-- 02_proc_fill_stats_iso_cursor.sql
-- ===== --

CREATE OR REPLACE PROCEDURE lr8_fill_priem_stats()
LANGUAGE plpgsql
AS $$
DECLARE
    v_подр int;
    v_дол int;
    v_всего int;
    v_совм int;
    cur_stats CURSOR FOR
```

```

SELECT
    пр.ид_подразделения,
    пр.ид_должности,
    COUNT(*)::int,
    COUNT(*) FILTER (WHERE пр.совместительство = B'1')::int
FROM прием пр
GROUP BY пр.ид_подразделения, пр.ид_должности
ORDER BY пр.ид_подразделения, пр.ид_должности;
BEGIN
DELETE FROM lr8_priem_stats;

OPEN cur_stats;
LOOP
    FETCH cur_stats INTO v_подр, v_дол, v_всего, v_совм;
    EXIT WHEN NOT FOUND;

    INSERT INTO lr8_priem_stats (ид_подразделения, ид_должности,
всего_приемов, совместителей)
        VALUES (v_подр, v_дол, v_всего, v_совм);
    END LOOP;
    CLOSE cur_stats;
END;
$$;

-- ===== --
-- 03_proc_update_where_current_of.sql
-- ===== --

CREATE OR REPLACE PROCEDURE lr8_reset_sovmest_old_priem()
LANGUAGE plpgsql
AS $$

```

```

DECLARE

v_ид int;
v_n int := 0;
cur_upd CURSOR FOR
    SELECT пр.ид
    FROM прием пр
    WHERE пр.дата_приема < DATE '2016-01-01'
    AND пр.совместительство = B'1'
    FOR UPDATE OF пр;
BEGIN
    OPEN cur_upd;
    LOOP
        FETCH cur_upd INTO v_ид;
        EXIT WHEN NOT FOUND;

        UPDATE прием
        SET совместительство = B'0'
        WHERE CURRENT OF cur_upd;

        v_n := v_n + 1;
    END LOOP;
    CLOSE cur_upd;

    RAISE NOTICE 'lr8: UPDATE WHERE CURRENT OF — изменено строк:
%', v_n;
END;
$$;

```

```

-- =====
-- 04_proc_delete_where_current_of.sql
-- =====

CREATE OR REPLACE PROCEDURE lr8_delete_demo_priem_cursor()
LANGUAGE plpgsql
AS $$
DECLARE
    v_ид int;
    v_n int := 0;
    cur_del CURSOR FOR
        SELECT пр.ид
        FROM прием пр
        WHERE пр.ид >= 900
        FOR UPDATE OF пр;
BEGIN
    OPEN cur_del;
    LOOP
        FETCH cur_del INTO v_ид;
        EXIT WHEN NOT FOUND;

        DELETE FROM прием
        WHERE CURRENT OF cur_del;

        v_n := v_n + 1;
    END LOOP;
    CLOSE cur_del;

    RAISE NOTICE 'lr8: DELETE WHERE CURRENT OF — удалено строк: %',
v_n;
END;

```

\$\$;

-- ===== --

-- 05_demo_cursor_types.sql

-- ===== --

DROP TABLE IF EXISTS _lr8_cursor_demo;

CREATE TEMP TABLE _lr8_cursor_demo (

тип_курсора text NOT NULL,

шаг text NOT NULL,

результат text NOT NULL

);

-- A) «Только вперёд» — NO SCROLL (аналог forward only / read only)

DO \$\$

DECLARE

r RECORD;

c CURSOR FOR

SELECT пд.ид FROM подразделение пд ORDER BY пд.ид;

v_first int;

v_second int;

BEGIN

OPEN c;

FETCH c INTO r;

v_first := r.ид;

FETCH c INTO r;

v_second := r.ид;

CLOSE c;

INSERT INTO _lr8_cursor_demo VALUES (

'NO SCROLL (ISO, только FETCH NEXT)',

```

        'первые два ид',
        v_first::text || ', ' || v_second::text
    );
END;
$$;

-- B) SCROLL — можно FETCH PRIOR (аналог scroll в T-SQL)
DO $$
DECLARE
    r RECORD;
    c SCROLL CURSOR FOR
        SELECT пд.ид FROM подразделение пд ORDER BY пд.ид;
    v1 int;
    v2 int;
    v_back int;
BEGIN
    OPEN c;
    FETCH NEXT FROM c INTO r;
    v1 := r.ид;
    FETCH NEXT FROM c INTO r;
    v2 := r.ид;
    FETCH PRIOR FROM c INTO r;
    v_back := r.ид;
    CLOSE c;

    INSERT INTO _lr8_cursor_demo VALUES (
        'SCROLL (ISO)',
        'NEXT, NEXT, PRIOR',
        'после PRIOR ид=' || v_back::text || ' (должен совпасть с ' || v1::text || ')'
    );

```


END;

\$\$;

-- C) «Статический» снимок — копия в TEMP на момент открытия (аналог
STATIC в T-SQL)

DO \$\$

DECLARE

r RECORD;

v_cnt_snap int;

v_cnt_live int;

c CURSOR FOR SELECT ш.ид FROM lr8_snap_static ш ORDER BY ш.ид;

BEGIN

DROP TABLE IF EXISTS lr8_snap_static;

CREATE TEMP TABLE lr8_snap_static AS

SELECT ш.ид, ш.количество_единиц FROM штатное_расписание ш;

v_cnt_snap := (SELECT COUNT(*) FROM lr8_snap_static);

OPEN c;

FETCH c INTO r;

CLOSE c;

UPDATE штатное_расписание SET количество_единиц =
количество_единиц WHERE ид = r.ид;

v_cnt_live := (SELECT COUNT(*) FROM штатное_расписание);

INSERT INTO _lr8_cursor_demo VALUES (

'STATIC (аналог через снимок TEMP)',

'курсор по снимку',

'строк в снимке: ' || v_cnt_snap::text || '; в живой таблице: ' || v_cnt_live::text

```

|| ' — изменения базы не меняют уже открытый снимок'
);
END;
$$;

-- D) «Динамический» — курсор напрямую по таблице (видит актуальные
данные при каждом FETCH)
DO $$
DECLARE
    r RECORD;
    v_before int;
    v_after int;
    c CURSOR FOR
        SELECT ш.количество_единиц FROM штатное_расписание ш WHERE
ш.ид = 1;
BEGIN
    SELECT ш.количество_единиц INTO v_before FROM штатное_расписание
ш WHERE ш.ид = 1;

    OPEN c;
        UPDATE    штатное_расписание    SET    количество_единиц    =
количество_единиц + 1 WHERE ид = 1;
    FETCH c INTO r;
    v_after := r.количество_единиц;
    CLOSE c;

        UPDATE    штатное_расписание    SET    количество_единиц    =
количество_единиц - 1 WHERE ид = 1;

    INSERT INTO _lr8_cursor_demo VALUES (

```

```

'DYNAMIC (аналог: курсор по живой таблице)',
'FETCH после UPDATE в той же сессии',
'было ' || v_before::text || ', FETCH видит ' || v_after::text
);
END;
$$;

-- E) FOR UPDATE — основа для WHERE CURRENT OF (в T-SQL близко к
keyset + scroll_locks)
DO $$
BEGIN
    INSERT INTO _lr8_cursor_demo VALUES (
        'KEYSET (в T-SQL) / FOR UPDATE (PostgreSQL)',
        'см. 03_proc_update_where_current_of.sql',
        'изменение текущей строки: UPDATE ... WHERE CURRENT OF'
    );
END;
$$;

-- === Результат выполнения скрипта (Data Output) ===
SELECT * FROM _lr8_cursor_demo ORDER BY тип_курсора, шаг;

SELECT
    'T-SQL STATIC' AS tsql,
    'снимок в tempdb, не видит чужие изменения' AS смысл
UNION ALL
SELECT 'T-SQL KEYSET', 'фиксированы ключи, данные строк могут
меняться'
UNION ALL
SELECT 'T-SQL DYNAMIC', 'каждый FETCH видит актуальные данные'

```

UNION ALL

SELECT 'PostgreSQL', 'NO SCROLL / SCROLL / снимок TEMP / FOR UPDATE

— см. демо выше';

-- ===== --

-- 06_trg_refresh_stats_cursor.sql

-- ===== --

CREATE OR REPLACE FUNCTION lr8_fn_refresh_priem_stats()

RETURNS trigger

LANGUAGE plpgsql

AS \$\$

DECLARE

 v_подр int;

 v_дол int;

 v_всего int;

 v_совм int;

 cur_ref CURSOR FOR

 SELECT

 пр.ид_подразделения,

 пр.ид_должности,

 COUNT(*)::int,

 COUNT(*) FILTER (WHERE пр.совместительство = B'1')::int

 FROM прием пр

 GROUP BY пр.ид_подразделения, пр.ид_должности;

BEGIN

 DELETE FROM lr8_priem_stats;

 OPEN cur_ref;

 LOOP

 FETCH cur_ref INTO v_подр, v_дол, v_всего, v_совм;

EXIT WHEN NOT FOUND;

INSERT INTO lr8_priem_stats (ид_подразделения, ид_должности,
всего_приемов, совместителей)

VALUES (v_подр, v_дол, v_всего, v_совм);

END LOOP;

CLOSE cur_ref;

RETURN NULL;

END;

\$\$;